02-614 Week 7
String Algorithms

Aidan Jan

October 20, 2026

Suffix Arrays (continued)

Suffix Array Construction

We will cover three algorithms with the following runtimes:

- $O(|T| \log^2 |T|)$: original
- $O(|T| \log |T|)$: original with a trick to reduce runtime
- $O(|T|)$: Skew Algorithm

Original Construction Algorithm

Suppose we have string $T = a_1 a_2 a_3 a_4 \ldots$. For example, let $T = cattcat\$\$$. (Note the double termius character)

Step 1

First, we will sort the suffixes by **their first two characters**, and assign each one a code number based on their ordering.

```
0 | $$
1 | attcat$
1 | at$
2 | cattcat$
2 | cat$
3 | t$
4 | tcat$
5 | ttcat$
```

- We only sort by the first two letters because if we sorted the entire suffix list normally it would take $O(|T|^2 \log |T|)$. Each string comparison takes worse case $O(|T|)$ time. However, if we only sort by the first two letters, then the string comparison is $O(1)$ because we always look at 2 characters. Therefore, the sorting time becomes $O(|T| \log |T|)$.

Step 2

Now, we will encode T using our new code. For our example,

$$T' = 21542130$$

Each number here is essentially a "supercharacter" of length 2. Note that from this, we are able to reconstruct the string since each number represents two characters (which then overlap).

Step 3

Sort the suffixes of T' by $(T'[i], T'[i+2])$. This is essentially sorting suffixes by the first 4 characters.

Step 4

Repeat. Assign a new code number to each of the suffixes of T', and repeat the process. This would then sort suffixes by 8 characters, then 16, etc. Once all the code numbers are different, we stop.

In total, we repeat $O(\log |T|)$ times. Therefore, our total time is $O(|T| \log^2 |T|)$ instead of $O(|T|^2 \log |T|)$.

Aside: Radix Sort

Forget strings for now. Let $A = [100, 123, 042, 333, 847, 892, 236]$. Radix sort places the numbers into buckets by their last digits.

```
0: 100
1:
2: 042, 892
3: 123, 333
4:
5:
6: 236
7: 847
8:
9:
```

Each bucket is a linked list. The order of the numbers in the bucket do not matter. Therefore, adding all the numbers into their buckets takes $O(n)$ time.

Now, we repeat this traversing the buckets in order, and sorting by the second-last digit into new buckets:

```
0: 100
1:
2: 123
3: 333, 236
4: 042, 847
5:
6:
7:
8:
9: 892
```

Observe that because we traversed the buckets from the first round in order, now the last two digits are in sorted order in this new set of buckets. Now, we just repeat this process with the third digit, etc.

The number of times we repeat is the length of the string. In the context of numbers, this would repeat $O(\log n)$ times, each time with n numbers. The total runtime for this sorting algorithm would be $O(n \log n)$.

Now, back to the suffix array construction algorithm, consider using Radix Sort with the codes for the numbers. This reduces the runtime to $O(|T| \log |T|)$ instead of $O(|T| \log^2 |T|)$.

Skew Algorithm

Step 0

First, we pad the string so the length $\equiv 1 \mod 3$. Suppose $T = mississippi\$$. We add a \$ to make it mod 3.

```
mis sis sip pi$ $
```

Now, we conceptually divide suffixes into three groups. $0 \mod 3, 1 \mod 3, 2 \mod 3$.

Step 1

We create `T' = concat(T[1:], T[2:])`

```
T' = iss iss ipp i$$ ssi ssi ppi
t =   C   C   B   A   E   E   D
```

Ignore the spaces, they are only there for clarity. The section before the \$\$ (each group of 3, that is) are group 1 suffixes. Meanwhile, each block afterwards are the group 2 suffixes.

Step 2

Encode T'.

- Use radix sort to sort the "codons"
- Recode T' using that code

Now, we have $t = CCBAEED$, where instead of using numbers for the code before, we use letters. (What we use really does not matter.)

Step 3

Repeat. If we recursively compute the suffix array for t, ignoring group 0 suffixes, then we slowly sort the array. Note that t is $\frac{2}{3}$ the length of T

Step 4

Compute the inverse suffix array of the array from step 3.

- An suffix array does the following: $A[i] = i$-th sorted suffix. Basically, given a number, return the suffix at that position.
- An inverse suffix array does the following: $S[i] =$ position in A of i. Basically, given a suffix, return when it appears.

Step 5

Sort group 0 suffixes using the key $(T[i], S_{i+1})$

Runtime

Skew algorithm runs in $O(|T|)$ time. In general, we can write the recursive relation:

$$T(n) \leq O(n) + T\left(\frac{2n}{3}\right)$$
$$\equiv T(n) \leq cn + T\left(\frac{2n}{3}\right)$$

Guess that $T(b) \leq 3cn$

$$\Rightarrow T(n) \leq cn + 3c\left(\frac{2n}{3}\right)$$
$$= cn + 2cn$$
$$= 3cn$$

In fact, we know that the algorithm runtime (and space complexity) scales linearly with 3x the length of the string.